

« DEVELOPMENT STANDARDS »

Development Processus

Document versions			
1.0	23/09/2024	COE_DEV_DEVELOPMENT PROCESS_230924_EN_V1.0	Creation
1.1	31/10/2024	COE_DEV_DEVELOPMENT PROCESS_311024_EN_V1.1	Modification
1.2	27/11/2024	COE_DEV_DEVELOPMENT PROCESS_271124_EN_V1.2	Validation

TABLE OF CONTENT

ABBREVIATIONS	3
OVERVIEW	4
GENERAL DESCRIPTION OF SCRUM APPROACH	4
TEAM VALUE AND PRINCIPALS	4
KEY ROLES	5
SPRINT ORGANIZATION.....	6
PLANNING	6
CEREMONIES	6
PROCESS TRACKING	7
DEVELOPMENT STANDARDS.....	9
CODE QUALITY	9
<i>Norms and coding conventions</i>	<i>9</i>
<i>Development and Code review.....</i>	<i>9</i>
TESTING PROCESS	10
<i>Unit Testing.....</i>	<i>10</i>
<i>Integration testing.....</i>	<i>10</i>
TESTING STANDARDS AND BEST PRACTICES.....	12
DEPLOYMENT PROCESS.....	13
TOOLS 13	
ENVIRONNEMENT DESCRIPTION.....	14
EXAMPLE OF DEPLOYMENT OF A SERVICE (MESSAGING V2).....	15

ABBREVIATIONS

PR	Pull Request
DOR	Definition Of Ready
DOD	Definition of Done
US	User Story
TS	Technical Story
CI	Continuous Integration

OVERVIEW

General description of scrum approach

Scrum is a management framework that teams use to self-organize and work towards a common goal. It describes a set of meetings, tools, and roles for efficient project delivery. Much like a sports team practicing for a big match, Scrum practices allow teams to self-manage, learn from experience, and adapt to change. Software teams use Scrum to solve complex problems cost effectively and sustainably.

Team value and principals

Certain principles and values characterize Scrum methodology:

Scrum principles for project success:

- Transparency

Teams work in an environment where everyone is aware of the challenges that others might be experiencing. Regular face-to-face conversations between cross-functional team members and project owners prevent miscommunication and information bottlenecks.

- Reflection

Frequent reflection points are built into the framework to allow team members to review their progress. Project managers use insights from these review meetings for estimation and future planning. As a result, projects can run more efficiently, within budget, and on schedule.

- Adaptation

Team members can reprioritize tasks based on changing customer requirements. They decide which tasks to complete first and which to revisit in the future.

Scrum Teams follow five core values.

- Commitment

Scrum Team members are committed to time-based tasks and goals and are dedicated to continuous improvement to find the best solution.

- Courage

Scrum Teams show courage by asking open, challenging questions. They have honest and transparent discussions to arrive at the best solution.

- Focus

During any given period, team members will work from a Product Backlog of tasks. They will focus on the selected tasks to provide deliverables within a limited time frame.

- Openness

Scrum Team members are open to new ideas and opportunities that support individual learning and overall project quality.

- Respect

Team members respect the project managers, each other, and the Scrum process. This culture of respect creates a spirit of mutual collaboration and cooperation within the team.

Key roles

A Scrum Team needs three specific roles: a Product Owner, Scrum leader, and development team.

Product Owner	The Product Owner focuses on ensuring the development team delivers the most value to the business. They understand and prioritize the changing needs of end users and customers. Effective product owners do the following: ➤ Give the team clear guidance on which features to deliver next. ➤ Bridge the gap between what the business wants and what the team understands. ➤ Decide when and how frequently releases should happen.
Scrum leader	Scrum leaders are the champions for Scrum within their teams. They are accountable for the Scrum Team's effectiveness. They coach teams, Product Owners, and the business to improve its Scrum processes and optimize delivery. Scrum leaders are also responsible for doing the following: ➤ Schedule the resources needed for each Sprint. ➤ Facilitate other Sprint events and team meetings. ➤ Lead digital transformation within the team. ➤ Facilitate any team training when adopting new technologies. ➤ Communicate with external groups to solve any challenges the team might be facing.
Scrum development team	The Scrum Team consists of testers, designers, UX specialists, Ops engineers, and developers. Team members have different skill sets and cross-train each other, so no one person becomes a bottleneck in delivering work. Jeff Bezos, the founder of Amazon, recommends the two-pizza rule when deciding team size. A team should be small enough to share two pizzas. Scrum development teams do the following: ➤ Work collaboratively to ensure a successful Sprint completion. ➤ Champion sustainable development practices. ➤ Self-organize and approach their projects with an evident we attitude. ➤ Drive the planning and estimating for how much work they can complete for each Sprint

SPRINT ORGANIZATION

Planning

- Sprint duration: 2 or 3 weeks
- DOR and DOD

In each sprint, DOR and DOD are addressed according with each ticket during a brief sprint planning session lasting 10 to 15 minutes to ensure their validation.

- **Example of DOR**
 - Technical and functional conception
 - Naming conventions for API
 - Dependencies are ready
- **Example of DOD**
 - If the ticket is developed and tests are OK
 - If There is a dependency with the domain or platform, the ticket is done if the PR is accepted, and the build is succeeded.
 - The ticket pass to done process if the PR is accepted the build is passed but there is an infrastructure issue.

Ceremonies

Agile ceremonies — also known as Scrum ceremonies or just ‘events’ — are specific events that provide a structured framework for iterative software development processes.

Sprint planning	These sessions are what initiate each sprint. Your team decides what work from the product backlog can be completed in the time, who will complete what tasks, and how you'll be successful.
Daily scrum (or daily stand-up)	These daily check-ups help teams stay on track and mark progress. Each morning, team members discuss what they worked on yesterday, what they're doing today, and what's blocking them from moving forward.
Sprint review	At the end of each sprint cycle, teams meet to demo what they've shipped and get early feedback from stakeholders. These review sessions can be informal 'show and tell' sessions or more formal meetings Champion sustainable development practices. ➤ Self-organize and approach their projects with an evident we attitude. ➤ Drive the planning and estimating for how much work they can complete for each Sprint
Sprint retrospective	At the end of each sprint cycle, the team also meets to discuss what's working (or not) on a process level, including blockers, communication issues, or a lack of resources.

Product backlog refinement

While not always listed as an 'official' ceremony, product backlog refinement is a continuous process of cleaning up, adjusting, and updating your task and feature list. Some teams set specific times to go through and clean up the backlog, while others do it continuously throughout each sprint.

Process tracking

When a new requirement arises, the Unibank team receives an EDB. The functional referent begins by framing and understanding the need with the client, then proceeds with the technical and functional design, validating the deliverable with the tech lead.

Afterward, a design authority meeting is scheduled to validate the requirement. This meeting involves architects, solution/security experts, infrastructure specialists, and enterprise architects to give the go-ahead for the design.

Next, the functional referent prepares the User Stories (US) and Technical Specifications (TS). The US includes a description of the context, the relevant APIs, technical and functional management rules, naming conventions (domain/database/swagger), acceptance criteria, and data sets if needed. The US is then assigned to the developer.

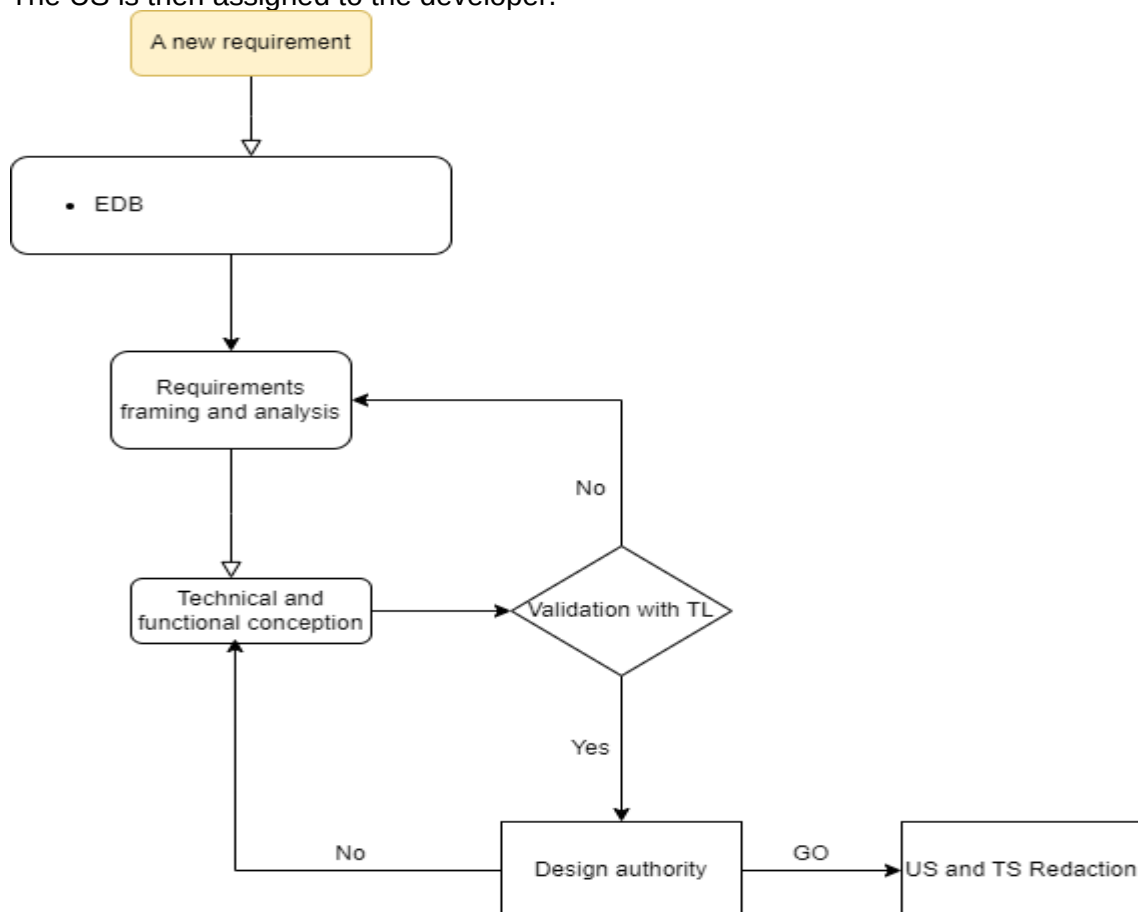


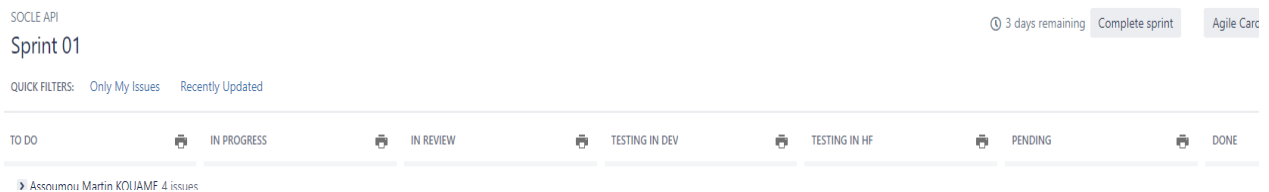
Figure 1: Project design and conception

DEVELOPMENT PROCESS

The development follows a sprint of Three weeks, features are described in user Stories.

Process description:

- **Creation:** A ticket is initially created in Jira. It starts in the “To Do” status.
- **Refinement:** If there are additional aspects to consider (such as estimation or description updates), the ticket can be refined. This might involve adding more details or clarifying requirements.
- **Ready:** Once the ticket is ready, it transitions to the “In Progress” status. At this point, a developer begins working on it.
- **Development:** The developer works on the ticket, making changes and improvements. When they’re done, they create a pull request (PR).
- **Code Review:** The ticket moves to the “Code Review” status. Other team member reviews the code changes in the PR. If the PR is accepted, the ticket proceeds to the next phase.
- **Validation:** the ticket pass to the status “validated” in the PR is accepted
- **Deployment:** After acceptance, the changes are deployed. The ticket moves to the “Testing” phase.
- **Testing:** Testing occurs in different environments. It starts in the development environment (dev env) and then progresses to higher environments (e.g., staging or production).



DEVELOPMENT STANDARDS

Code quality

Norms and coding conventions

(Cf. norms and conventions of coding)

Development and Code review

The developer starts working on the assigned ticket. During the development phase, they can communicate with the Business Analyst (BA) and the tech lead for functional or technical specifications.

When the developer finishes working on the project, they create a Pull Request (PR) for code review and validation before deployment.

1. Preparation:

The developer completes their code changes and ensures the code is ready for review.

They create a Pull Request (PR) in the version control system, detailing the changes made and the context.

2. Review Request:

The developer assigns the PR to one or more peers for review. This can include team members, tech leads, or other relevant stakeholders.

3. Review:

Reviewers examine the code for various aspects, including:

- **Functionality:** Ensuring the code works as intended and meets the requirements.
- **Code Quality:** Checking for readability, maintainability, and adherence to coding standards.
- **Performance:** Assessing if the code performs efficiently.
- **Security:** Identifying any potential security vulnerabilities.
- **Testing:** Verifying that appropriate tests are included and pass successfully.

4. Feedback:

Reviewers provide feedback, which can include comments, suggestions, and requests for changes. The feedback is documented within the PR/MR for transparency and tracking.

5. Revisions:

The developer addresses the feedback by making necessary changes to the code.

They update the PR with the revised code and may add comments explaining the changes.

6. Approval:

Once the reviewers are satisfied with the changes, they approve the PR. Multiple approvals may be required depending on the project's guidelines.

7. **Merge:**

After approval, the code is merged into the main branch. Automated tests and continuous integration (CI) pipelines may run to ensure the integration is smooth and does not break existing functionality.

Testing process

Unit Testing

- **Env** : local
- **Responsible:** Developer
- **Description:**

Unit testing is a type of software testing where individual units or components of a software are tested in isolation from other parts of the system. The goal is to validate that each unit of the software performs as expected.

- **Purpose:**
 - Verify Correctness: Ensure that each function or method produces the correct output for a given input.
 - Identify Bugs Early: Detect and fix bugs early in the development process, reducing the cost and effort of fixing them later.
 - Facilitate Changes: Make it easier to change or refactor code. If unit tests exist, developers can confidently make changes knowing that they will be alerted if something breaks.
 - Improve Design: Encourage better software design by making developers think about the code's interface and behavior in isolation.

Integration testing

- **Env** : local
- **Responsible** : Developer
- **Description**

Integration testing is a type of testing in which individual software modules are combined and tested as a group. It aims to verify that the integrated components function correctly together.
- **Purpose:**

Detect Interface Issues: Identify problems in the interfaces and interaction between different modules or components.

 - Verify Data Flow: Ensure that data is correctly passed between modules.
 - Validate Combined Behavior: Confirm that the combined behavior of the integrated components meets the specified requirements.
- **Types of Integration Testing:**

- Top-Down Integration Testing: Starts testing from the top of the module hierarchy and progresses downward, often using stubs for lower-level modules.
- Bottom-Up Integration Testing: Begins testing from the bottom of the module hierarchy and works upwards, using drivers to simulate higher-level modules.
- Sandwich (Hybrid) Integration Testing: Combines both top-down and bottom-up approaches to test high-level and low-level modules simultaneously.
- Big Bang Integration Testing: Integrates all components simultaneously and tests them. This approach can be risky if issues are found, as it can be harder to isolate the cause.

- **Process:**

- Identify Modules: Determine which modules or components need to be integrated and tested together.
- Create Test Cases: Develop test cases that focus on the interactions between the integrated components.
- Prepare Test Environment: Set up the necessary environment, including stubs or drivers if needed.
- Execute Tests: Run the integration tests and monitor the interactions between the modules.
- Record Results: Document the outcomes of each test case, noting any issues or failures.
- Debug and Fix: Address any identified problems and re-test to ensure issues are resolved.

TESTING STANDARDS AND BEST PRACTICES

Type of testing	Best practices
Unit Testing	▪ Isolation: Test units in isolation by mocking dependencies. This ensures tests are independent and reliable. ▪ Automation: unit tests are a part of the continuous integration (CI) pipeline, ensuring tests run automatically with each code change. ▪ Coverage: Aim for high test coverage but prioritize meaningful tests that cover critical paths and edge cases. In our projects, the test coverage should be at least 85% ▪ Maintainability: Write clear, maintainable tests. Good tests are easy to read and understand, making it straightforward for others to see what is being tested and why.
Integration Testing	▪ Start Small and Build Up: Begin with testing smaller, closely related modules before moving on to larger integrations. This incremental approach helps isolate issues more effectively. ▪ Test Data Management: Maintain a consistent and reliable set of test data. Ensure that the test data covers various scenarios, including edge cases and error conditions. ▪ Mock External Dependencies: Use mocking frameworks (e.g., Mockito for Java) to simulate external dependencies, such as third-party APIs. This helps test the integration without relying on external systems. ▪ Verify Data Integrity: Ensure that data is correctly passed between components and maintains its integrity. This includes checking for data corruption and unexpected transformations. ▪ Handle Exceptions and Errors: Ensure that your integration tests handle exceptions and errors gracefully. Test how the system behaves under failure conditions and verify that appropriate error messages are generated.

DEPLOYEMENT PROCESS

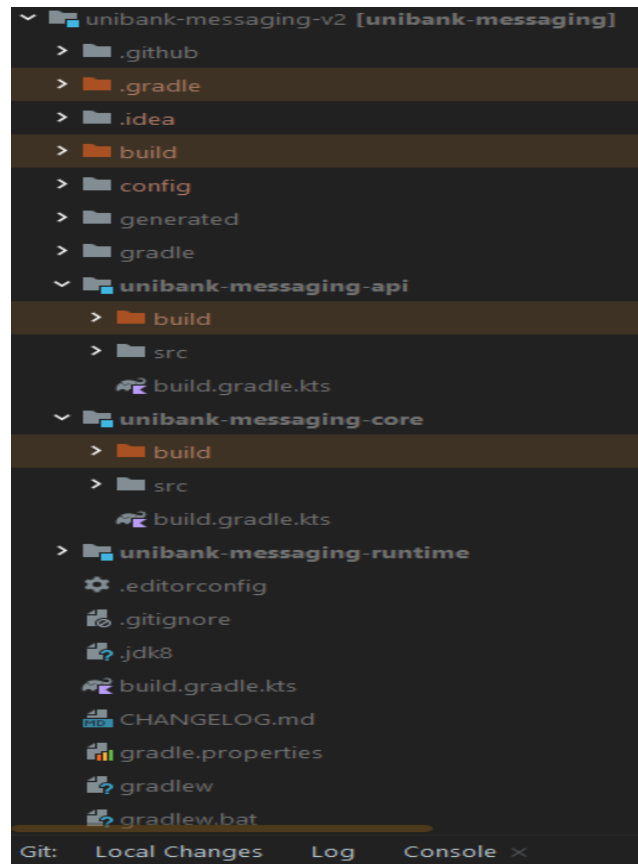
Tools

Git	Control version	https://sgithub.fr.world.socgen/SGABS/
Sonar	Sast	https://dsp-sonar.fr.world.socgen/sonar/projects
Nexus itbox	Artifactory	https://itbox-nexus-arc.fr.world.socgen/nexus-arc/#browse/browse:sgcib-sgabs-maven2-hosted-releases:com%2Fsocgen%2Funibank%2Fv2%2Funibank-messaging-runtime%2F2.0.530%2Funibank-messaging-runtime-2.0.530.jar
Jenkins CDP	Job orchestrator	https://cdp-jenkins-paas-xsf.fr.world.socgen/login?from=%2Fjob%2FABB%2Fjob%2FAPPS%2F
Consul	Service discovery	https://consul-uat.afs.socgen/ui/dc1/kv
Jenkins sofa	Job orchestrator – to be decommissioned	https://sofa.dns20.socgen/cloudbees/login?from=%2Fcloudbees%2Foperations-center-sso%2Fauthenticate%3Fid%3Df9bc2660-09ea-4965-95a3-6c0d6ece844f%26from%3Dhttps%3A%252f%252fsofa.dns20.socgen%252fjenkins-abs%252fsecurityRealm%252ffinishLogin
Nexus sofa	To be decommissioned	https://sofa.dns20.socgen/nexus/

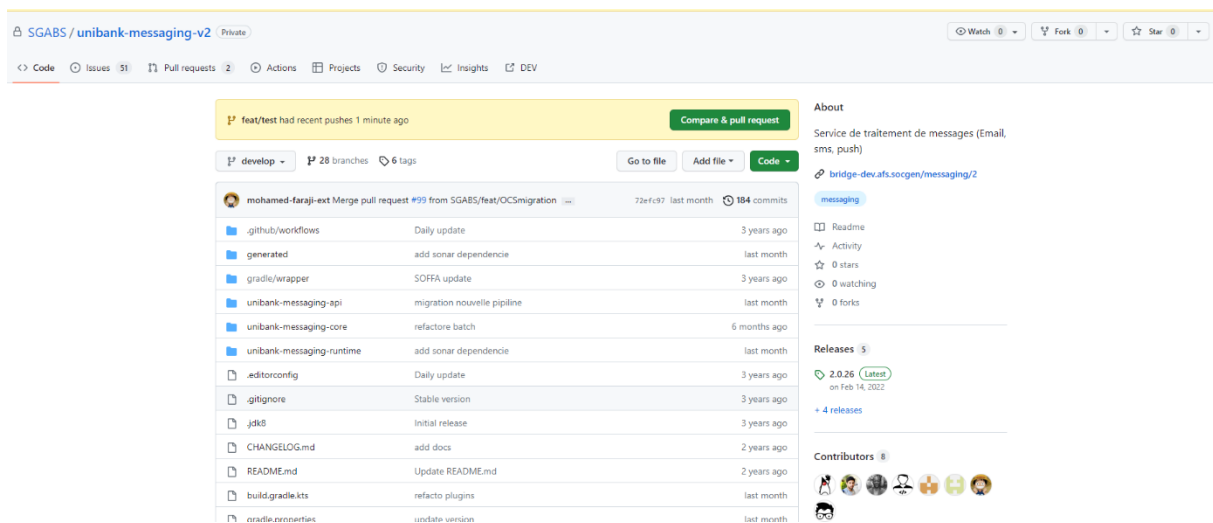
Environnement description

DEV	Env for integration testing	• Dev environment is not accessible for all projects (Example OpenR) • Running a pipeline or creating a ticket for DevOps team if you don't have access to the pipeline
HF	Functional homologation	Running a pipeline or creating a ticket for DevOps team if you don't have access to the pipeline
HT	Technical homologation	A ticket is created to DevOps team to lunch the deployment
PROD	Production env	Production deployment process

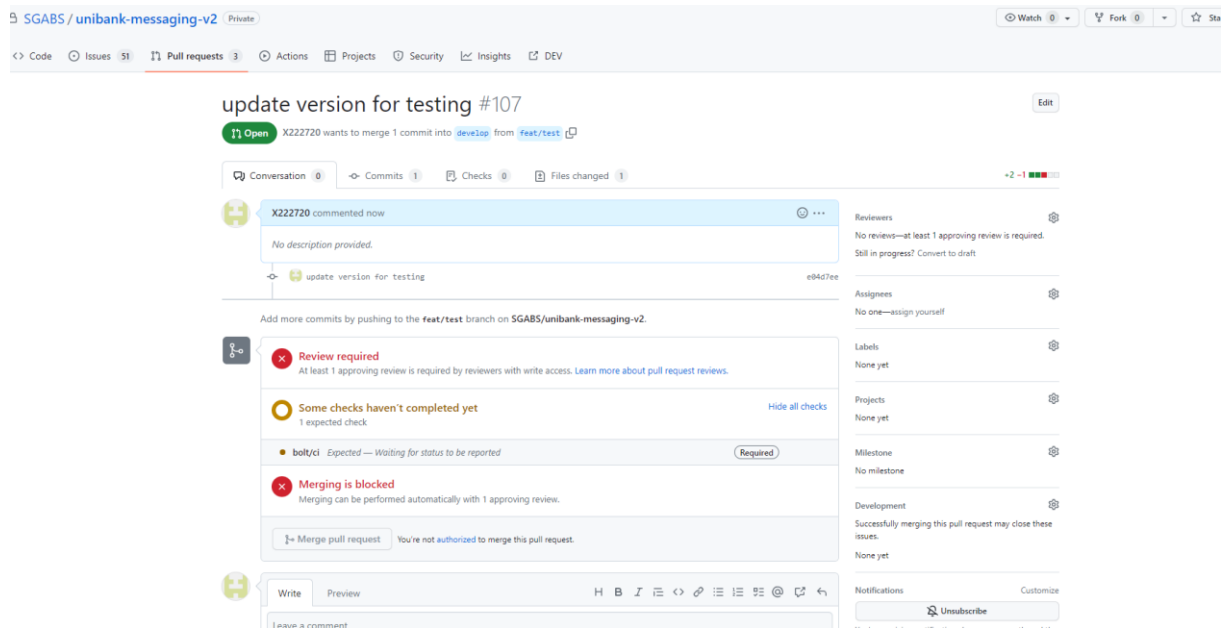
Example of deployment of a service (messaging V2)



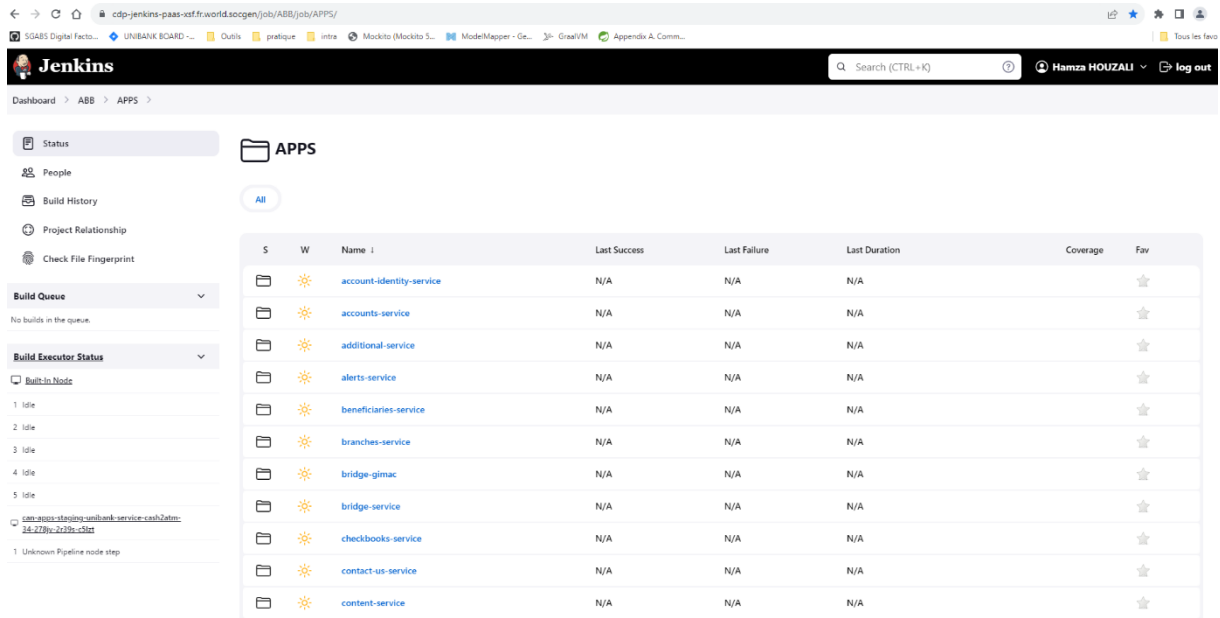
- Step 1: build the project in your local development and make sure the build is successful after the dev.
- Step 2: Access to sgit and create the PR, make sure to mention the teach lead or another peer to review the request and approve it.

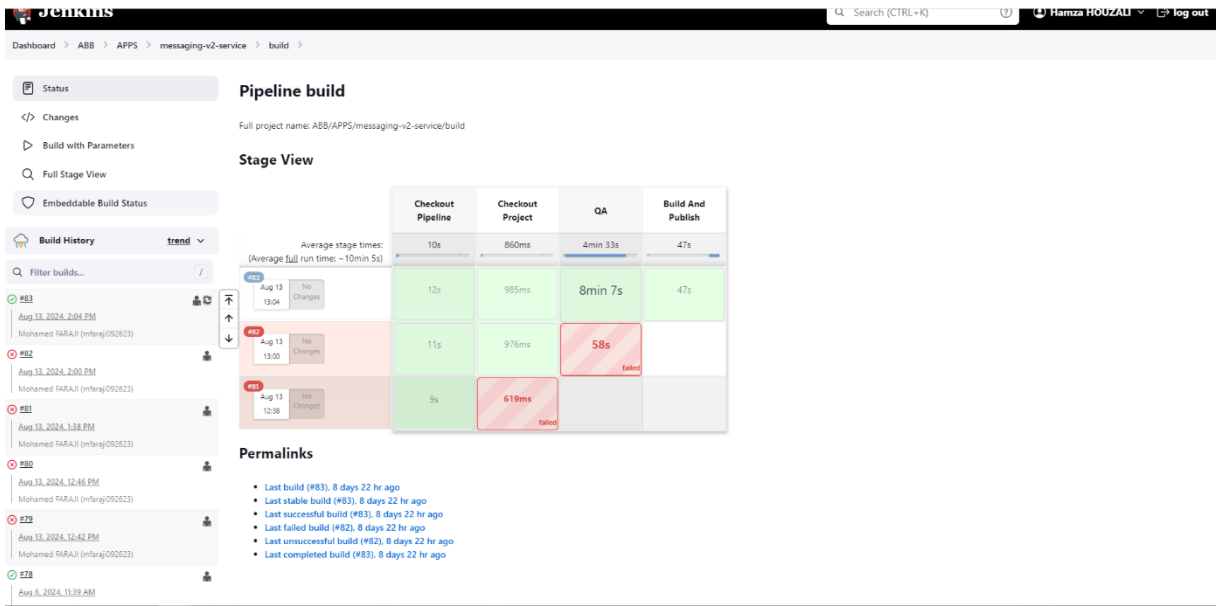


- Step 3: Access to sgit and create the PR, make sure to mention the teach lead or another peer to review the request and approve it.



- Step 4: access to Jenkins and run the job of building. Make sure the build succeeds and get the jar version.





Jenkins Search (CTRL+K) Hamza HOUZAU log out

Dashboard > ABB > APPS > messaging-v2-service > build

Status Changes Build with Parameters Full Stage View Embeddable Build Status

Pipeline build
Full project name: ABB/APPS/messaging-v2-service/build

Stage View

	Checkout Pipeline	Checkout Project	QA	Build And Publish
Avg (Average full run time: ~10min 5s)	10s	860ms	4min 33s	47s
#83 Aug 13 13:04 No Changes	12s	985ms	8min 7s	47s
#82 Aug 13 13:00 No Changes	11s	976ms	58s failed	
#81 Aug 13 12:58 No Changes	9s	618ms failed		

Permalinks

- Last build (#83), 8 days 22 hr ago
- Last stable build (#83), 8 days 22 hr ago
- Last successful build (#83), 8 days 22 hr ago
- Last failed build (#82), 8 days 22 hr ago
- Last unsuccessful build (#82), 8 days 22 hr ago
- Last completed build (#83), 8 days 22 hr ago

Build History trend

Filter builds...

- #83 Aug 13, 2024, 2:04 PM Mohamed FARAJ (mfaraj@092623)
- #82 Aug 13, 2024, 2:00 PM Mohamed FARAJ (mfaraj@092623)
- #81 Aug 13, 2024, 1:58 PM Mohamed FARAJ (mfaraj@092623)
- #80 Aug 13, 2024, 12:46 PM Mohamed FARAJ (mfaraj@092623)
- #79 Aug 13, 2024, 12:42 PM Mohamed FARAJ (mfaraj@092623)
- #78 Aug 8, 2024, 11:39 AM

messaging-v2-service > build

Pipeline build

This build requires parameters:

GIT_REF
Branch to build
develop

☒ QA
Run QA analysis

☒ NEXUS
Release artifact (version will be the Jenkins BUILD_NUMBER)

☐ DOCKER

Build Cancel

```

Dashboard > ABB > APPS > messaging-v2-service > build > #83

Cannot upload checksum for module-maven-metadata.xml because the remote repository doesn't support SHA-256. This will not fail the build.

> Task :unibank-messaging-core:publish
> Task :unibank-messaging-runtime:compileJava UP-TO-DATE
> Task :unibank-messaging-runtime:processResources UP-TO-DATE
> Task :unibank-messaging-runtime:classes UP-TO-DATE
> Task :unibank-messaging-runtime:bootJarMainClassName UP-TO-DATE
> Task :unibank-messaging-runtime:bootJar UP-TO-DATE
> Task :unibank-messaging-runtime:jar SKIPPED
> Task :unibank-messaging-runtime:generateMetadataFileForMavenPublication
> Task :unibank-messaging-runtime:generatePomFileForMavenPublication

> Task :unibank-messaging-runtime:publishMavenPublicationToMavenRepository
Cannot upload checksum for module-maven-metadata.xml because the remote repository doesn't support SHA-256. This will not fail the build.
Cannot upload checksum for module-maven-metadata.xml because the remote repository doesn't support SHA-512. This will not fail the build.

> Task :unibank-messaging-runtime:publish

Deprecated Gradle features were used in this build, making it incompatible with Gradle 8.0.

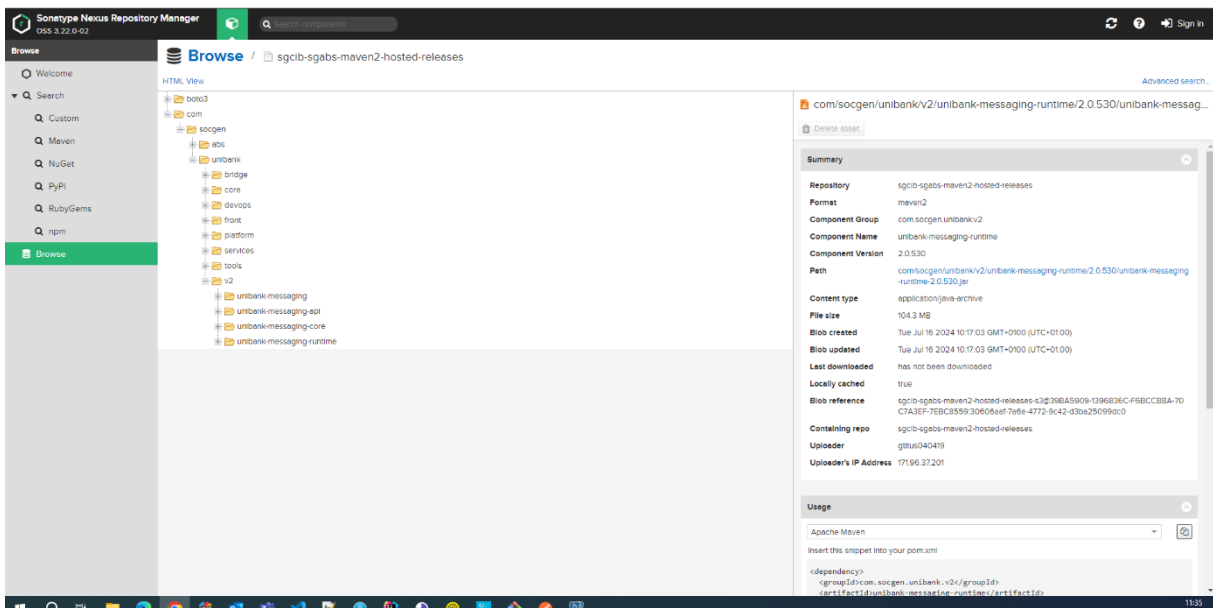
You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

See https://docs.gradle.org/7.6.1/userguide/command_line_interface.html#sec:command_line_warnings

BUILD SUCCESSFUL in 32s
17 actionable tasks: 9 executed, 8 up-to-date
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // container
[Pipeline] container
[Pipeline] {
[Pipeline] sh
+ grep '^version=' /home/jenkins/agent/workspace/ABB/APPS/messaging-v2-service/build/project/gradle.properties
+ cut -d= -f2
[Pipeline] withEnv
[Pipeline] {

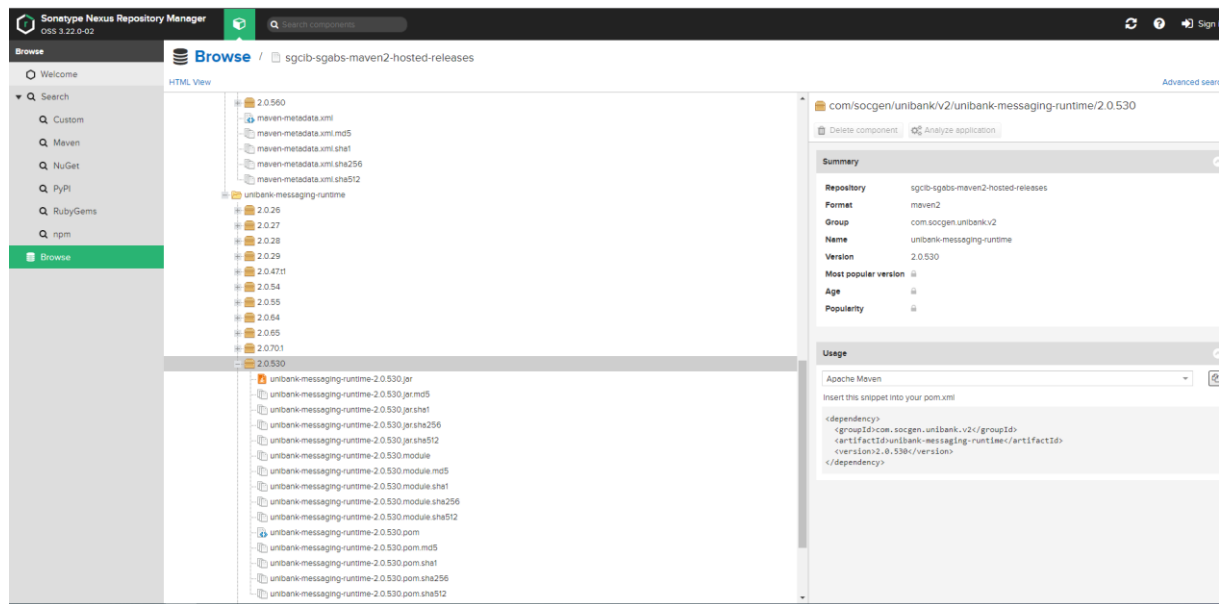
```

- Step 5: verify the jar is deposited in nexus.

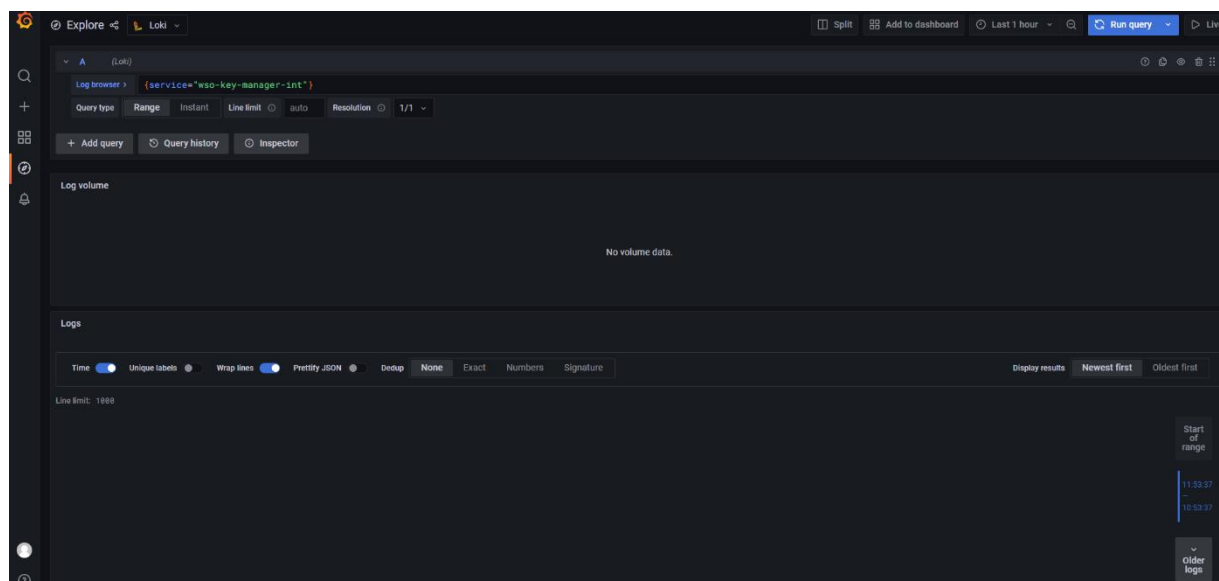


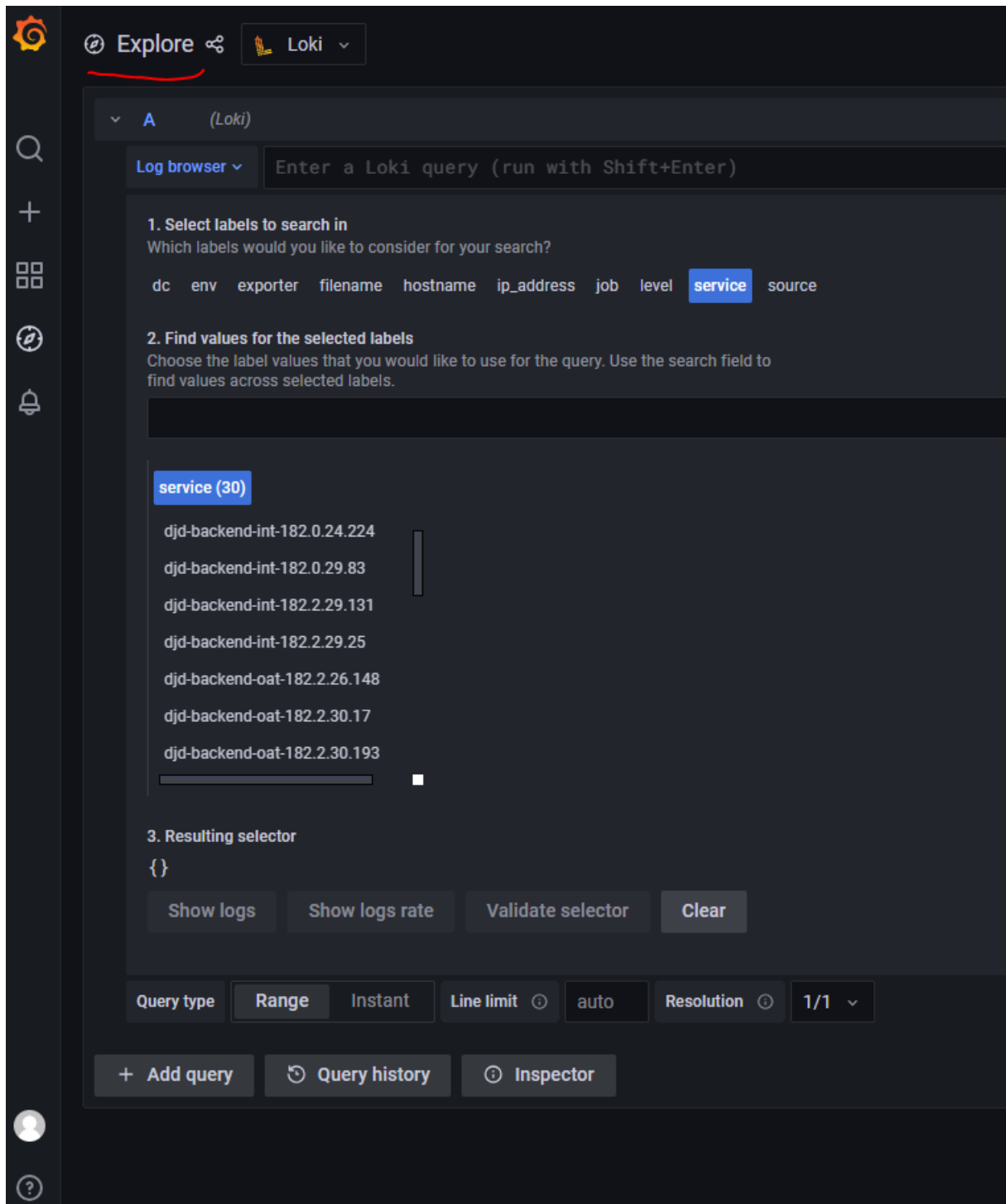
The screenshot shows the Sonatype Nexus Repository Manager interface. On the left, a sidebar lists various package managers. The main area displays a tree view of the repository structure, with the path `com/socgen/unibank/v2/unibank-messaging-runtime` selected. On the right, a detailed view of the artifact `com/socgen/unibank/v2/unibank-messaging-runtime/2.0.530/unibank-messaging-runtime-2.0.530.jar` is shown. The summary section includes details such as Repository, Format, Component Group, Component Name, Component Version, Path, Content type, File size, Blob created, Blob updated, Last downloaded, Locally cached, Blob reference, Containing repo, Uploader, and Uploader's IP Address. The Usage section shows the artifact being used in a Maven project, with a snippet of the `pom.xml` file.

- Step 6:
 - If you are in a dev env or HF you can get the version and deploy using Jenkins pipeline
 - for the HT environments, create a ticket for DevOps team to deploy the specific jar.



- Step 7: You can verify the logs using Grafana.





The screenshot shows the Grafana Explore interface with the Loki data source selected. The interface is divided into three main sections for building a query:

- 1. Select labels to search in**
Which labels would you like to consider for your search?
Labels: dc, env, exporter, filename, hostname, ip_address, job, level, **service**, source.
- 2. Find values for the selected labels**
Choose the label values that you would like to use for the query. Use the search field to find values across selected labels.
Results for **service (30)**:
 - djd-backend-int-182.0.24.224
 - djd-backend-int-182.0.29.83
 - djd-backend-int-182.2.29.131
 - djd-backend-int-182.2.29.25
 - djd-backend-oat-182.2.26.148
 - djd-backend-oat-182.2.30.17
 - djd-backend-oat-182.2.30.193
- 3. Resulting selector**
{}

Buttons at the bottom of the selector section: Show logs, Show logs rate, Validate selector, Clear.

Query configuration at the bottom:
Query type: Range | Instant | Line limit: auto | Resolution: 1/1

Navigation buttons: + Add query, Query history, Inspector.

